🚀 7天 Linux 高性能网络编程（Epoll）进阶实战计划

本计划旨在帮助具有 C++ 业务开发背景的工程师，从应用层向下钻研，掌握 Linux 系统调用、IO 多路复用及其高性能演进逻辑。

🗓️ 第 1 天：TCP/IP 协议栈与 Socket 基础

核心目标： 建立对“流”的敬畏，理解连接的生命周期。

1. 理论必会

- **三次握手/四次挥手：** 能够手绘状态转换图，理解 `SYN_SENT` , `ESTABLISHED` , `FIN_WAIT_2` , `TIME_WAIT` 。
- **关键参数：**
  - `TIME_WAIT` 的作用是什么？为什么是 2MSL？
  - `listen` 函数中的 `backlog` 是什么？理解半连接队列（SYN Queue）与全连接队列（Accept Queue）。

2. 实战任务

- **任务：** 编写一个最基础的 C/S 模型。
  - **Server：** `socket() -> bind() -> listen() -> accept() -> recv() -> send()` 。
  - **Client：** `socket() -> connect() -> send() -> recv()` 。
- **要求：** 纯手工编写，不准复制。
- **思考：** 当 Server 正在 `recv` 阻塞时，第二个客户端尝试 `connect` 会发生什么？

🗓️ 第 2 天：阻塞 vs 非阻塞 IO

核心目标： 理解为什么业务代码在底层必须是“非阻塞”的。

1. 理论必会

- **阻塞 (Blocking)：** 进程挂起，等待内核数据准备就绪。
- **非阻塞 (Non-blocking)：** `fcntl(fd, F_SETFL, O_NONBLOCK)` ，内核立即返回。
- **EAGAIN / EWOULDBLOCK：** 理解这个错误码的含义——“现在没数据，晚点再来”。

2. 实战任务

- **任务：** 修改第 1 天的代码，将 Socket 设置为 `O_NONBLOCK` 。
- **观察：** 如果不使用任何多路复用技术，直接写一个 `while(true) { recv(...) }` 循环，CPU 占用率会发生什么变化？
- **挑战：** 尝试在 C++ 中封装一个 `Socket` 类，利用 **RAII** 机制在析构函数中自动 `close(fd)` 。

🗓️ 第 3 天：多路复用的先驱：Select 与 Poll

核心目标： 见识“旧时代的局限性”，为理解 Epoll 埋下伏笔。

1. 理论必会

- **Select：** 最大 1024 限制，内核/用户态位图拷贝开销，$O(n)$ 的轮询复杂度。
- **Poll：** 解决了 1024 数量限制，但本质依然是轮询。

2. 实战任务

- **任务：** 编写一个基于 `select()` 的多客户端回显服务器。
- **思考：** 为什么每次调用 `select` 之前，都要重新初始化 `fd_set`？这在性能上意味着什么？

🗓️ 第 4 天：Epoll 核心机制（重难点）

核心目标： 彻底搞懂 Epoll 为什么快，以及它的两种“脾气”。

1. 理论必会

- **三大 API：** `epoll_create`（内核红黑树），`epoll_ctl`（增删改），`epoll_wait`（等待就绪链表）。
- **触发模式：**
  - **LT (Level Triggered)：** 水平触发（只要有水就提醒）。
  - **ET (Edge Triggered)：** 边缘触发（水位变化才提醒）。
- **数学模型：** 理解为什么 Epoll 是 $O(1)$ 的事件触发。

2. 深度思考

- **为什么 ET 模式必须配合非阻塞 IO 和 `while` 循环？**（如果不一次性读完，下次 `wait` 将不再提醒）。

🗓️ 第 5 天：手写 Epoll 高并发服务器

核心目标： 将零散的 API 组合成一个可以处理成千上万连接的雏形。

1. 实战任务

- **任务：** 实现一个支持多并发连接的 Epoll Server。
- **功能要求：**
  - 使用 `std::map<int, ConnectionContext>` 管理每个 FD 的私有数据。
  - 处理客户端正常关闭（`recv` 返回 0）和异常关闭（`ECONNRESET`）。
  - 能够响应 `EPOLLIN`（读事件）并正确处理 `EPOLLOUT`（写事件）。

🗓️ 第 6 天：Reactor 模型与线程策略

核心目标： 从“写代码”升华为“架构设计”。

1. 理论必会

- **Reactor 模式：** 反应堆模式。区分“事件分发器”、“处理器”、“事件源”。
- **线程模型：**
  - **单 Reactor 单线程：** 简单，但不适合重计算任务。
  - **主从 Reactor (Main + Sub)：** Nginx/Muduo 模型。主 Reactor 只负责 `accept`，子 Reactor 负责处理 IO。

2. 架构练习

- **思考：** 如果业务逻辑（比如计算 MD5 或查数据库）很慢，应该放在 Reactor 线程里吗？如何设计一个简单的任务队列投递到线程池？

📅 第 7 天：面试强化与自我审阅

核心目标： 能够清晰、专业地向面试官或同事描述这套机制。

1. 自测问题清单

- [ ] 讲一下 Epoll 底层的红黑树和就绪链表是如何协作的？
- [ ] 如果监听的 FD 被关闭了，Epoll 会自动将其移除吗？
- [ ] 什么是“惊群效应”？在高版本内核中是如何解决的？
- [ ] 为什么 ET 模式在高并发场景下更被推荐？
- [ ] 什么是 `SIGPIPE` 信号？在写网络程序时如何处理它？

2. 进阶建议

- 阅读一份成熟的开源库源码（推荐 **Muduo** 或 **Libevent**）。
- 尝试通过 `ss -ant` 命令观察不同状态下的连接。